

DOCUMENT 6 — QUALIFYAI PROJECT DOCUMENTATION & REFERENCE STRUCTURE

Document Type: Project Documentation, Engineering Reference & Development Organization

Project: QualifyAI

Purpose: Define how QualifyAI's formal documents, technical references, engineering specifications, project state, and supporting documentation should be organized and maintained.

1. Document Overview

QualifyAI is a multi-layer technical product containing:

- Public product experience
- Recruiter application
- Candidate interview application
- Real-time voice infrastructure
- AI integrations
- Technical evaluation
- Communication analysis
- Assessment integrity
- Multi-tenant data architecture
- Reporting and analytics

Because the platform contains multiple interconnected engineering domains, documentation must remain structured and traceable.

This document defines the documentation architecture that connects:

PRODUCT REQUIREMENTS



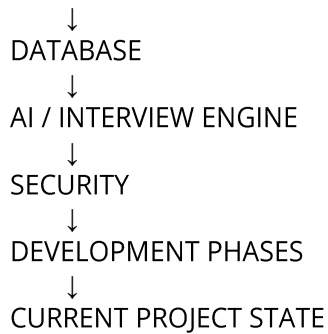
SYSTEM ARCHITECTURE



DESIGN



IMPLEMENTATION



2. Documentation Philosophy

QualifyAI documentation should follow one central principle:

Every major product decision should have a clear documentation home.

Documentation should not become a collection of duplicated notes.

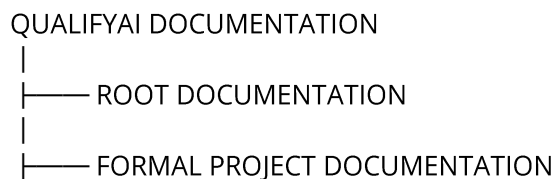
Each document should have a specific responsibility.

The documentation system should answer:

- What is QualifyAI?
- What should it do?
- How is it architected?
- How should it look and behave?
- How should it be implemented?
- What has already been implemented?
- What should be implemented next?
- What rules must developers follow?

3. Documentation Layers

QualifyAI documentation is divided into three major layers:



|
└── ENGINEERING / SUPPORTING DOCUMENTATION

These correspond to:

Root Markdown Files

↓
document/
↓
docs/

4. Root Documentation

The root contains seven permanent Markdown documents:

QUALIFYAI/
|
├── README.md
├── prd.md
├── architecture.md
├── design.md
├── memory.md
├── phases.md
└── rules.md

These documents form the project's permanent source-of-truth layer.

5. README.md

README.md is the entry point for the entire project.

It should provide a concise but complete understanding of QualifyAI.

Recommended contents

README.md
|
├── Project Overview
├── Problem
├── Solution
├── Core Features
└── Architecture Overview

└──	Technology Stack
└──	Repository Structure
└──	Development Overview
└──	Documentation Index

The README should answer:

What is QualifyAI and where should a developer start?

6. prd.md

prd.md is the product requirements source of truth.

It should contain:

- Product vision
- Problem statement
- Target users
- User roles
- Product workflows
- Functional requirements
- Non-functional requirements
- Feature requirements
- Product scope
- Current scope
- Future scope
- Acceptance criteria

Its relationship is:

```
USER NEED
↓
PRODUCT REQUIREMENT
↓
FEATURE
↓
ACCEPTANCE CRITERIA
```

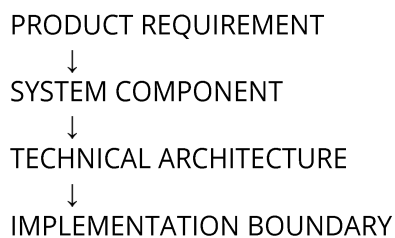
7. architecture.md

architecture.md defines the technical architecture.

It should describe:

- System architecture
- Client architecture
- Server architecture
- Database architecture
- AI architecture
- Real-time architecture
- Evaluation architecture
- Proctoring architecture
- Authentication
- Multi-tenancy
- Security
- Deployment
- External integrations

The architectural flow is:



8. design.md

design.md is the design-system and UX source of truth.

It should contain:

- Visual principles
- Design language
- Layout principles
- Typography
- Color system
- Component principles
- Interaction behavior
- Responsive rules

- Accessibility principles
- Recruiter UX
- Candidate UX
- Interview UX

The design document should remain separate from backend implementation details.

9. memory.md

memory.md is the living project-state document.

It should track:

CURRENT STATE
↓
COMPLETED WORK
↓
ACTIVE WORK
↓
BLOCKERS
↓
IMPORTANT DECISIONS
↓
NEXT STEPS

It should be updated as implementation progresses.

It should not become a duplicate of the PRD or architecture document.

10. phases.md

phases.md defines the development roadmap.

The documented implementation direction can be represented as:

PHASE 1
Foundation & Authentication
↓
PHASE 2
JD Parser & Rubric

↓
PHASE 3
Real-Time Speech Pipeline
↓
PHASE 4
Proctoring & Analytics
↓
PHASE 5
Optimization, Security & Deployment

Each phase should define:

- Objective
- Dependencies
- Deliverables
- Acceptance criteria
- Completion state

11. rules.md

rules.md defines engineering invariants.

It should cover:

Code Rules

- Separation of responsibilities
- Naming conventions
- Module boundaries
- Reusability

Security Rules

- Authentication
- Authorization
- Tenant isolation
- Secret handling
- Data access

AI Rules

- AI provider abstraction
- Output validation
- Evaluation boundaries
- Failure handling

Product Rules

- Current scope
- Unsupported functionality
- Future-scope boundaries

12. Formal Project Documentation

The document/ directory is separate from living engineering documentation.

```
document/  
|  
├── Formal Project Artifacts  
├── Academic Documentation  
├── Approved Specifications  
├── Architecture References  
└── Project Deliverables
```

The purpose of this directory is to preserve finalized or formal project documents.

It should not become a temporary scratchpad.

13. Engineering Documentation

The docs/ directory contains supporting technical documentation.

Conceptually:

```
docs/  
|  
├── API  
├── AI  
├── Interview Engine  
└── Database
```

- Security
- Integrations
- Deployment
- Development
- Technical Decisions

The exact documents can grow as the implementation develops.

14. Documentation Ownership

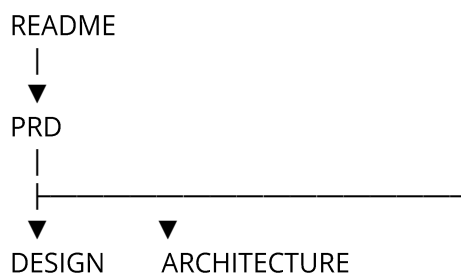
Each major concern has one primary source of truth.

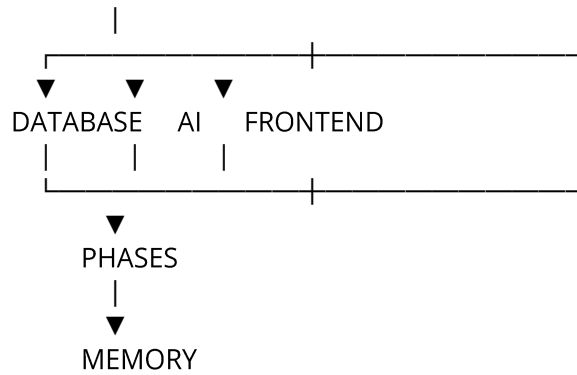
Concern	Primary Document
Product definition	prd.md
Technical architecture	architecture.md
Visual/UX design	design.md
Current implementation state	memory.md
Development roadmap	phases.md
Engineering rules	rules.md
Project introduction	README.md
Formal deliverables	document/
Detailed technical references	docs/

This prevents conflicting specifications.

15. Documentation Dependency Model

The documents are interconnected.





16. Product-to-Implementation Traceability

A feature should be traceable from product definition to implementation.

Example:

```
FEATURE
AI Technical Interview
↓
PRD
↓
Architecture
↓
Frontend Interview Room
↓
WebSocket Interview Session
↓
STT
↓
LLM
↓
TTS
↓
Transcript
↓
Evaluation
↓
Report
```

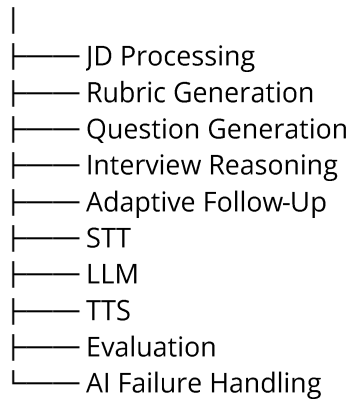
This makes the architecture auditable and easier to maintain.

17. AI Documentation Structure

AI is one of the central components of QualifyAI and should have dedicated supporting documentation.

Conceptually:

AI DOCUMENTATION



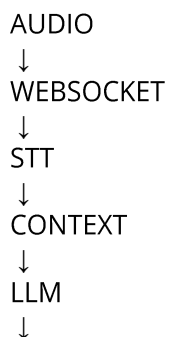
Each AI component should document:

- Purpose
- Input
- Output
- Dependencies
- Validation
- Failure behavior
- Security considerations

18. Interview Engine Documentation

The real-time interview engine deserves its own engineering documentation.

The conceptual flow is:



NEXT ACTION



TTS



AUDIO

The documentation should define:

- Session lifecycle
 - Connection states
 - Audio handling
 - Transcript flow
 - Context management
 - Follow-up generation
 - Error recovery
 - Completion behavior
-

19. Evaluation Documentation

Evaluation documentation should define how interview data becomes scores.

TRANSCRIPT



RUBRIC



ANSWER ANALYSIS



TECHNICAL EVALUATION



COMMUNICATION ANALYSIS



SCORE

It should document:

- Evaluation dimensions
- Rubric relationship
- Score generation
- Evidence
- Communication metrics
- Report generation
- Evaluation failure handling

20. Proctoring Documentation

Proctoring documentation should define:

EVENT
↓
DETECTION
↓
CLASSIFICATION
↓
RISK SIGNAL
↓
SUMMARY

Documented signals include:

- Focus loss
- Tab switching
- Browser visibility changes
- Acoustic anomalies
- Possible secondary voices

QualifyAI should deliberately avoid making facial micro-expression analysis a core scoring mechanism.

21. Database Documentation

Database documentation should cover:

- Entity relationships
- Tenant architecture
- Ownership
- Authentication mapping
- RLS
- Indexing
- Data lifecycle
- Storage relationships
- Migration strategy

The conceptual relationship is:

USER
↓
ORGANIZATION
↓
JOB
↓
CANDIDATE
↓
INTERVIEW
↓
EVALUATION
↓
REPORT

22. API Documentation

API documentation should describe:

CLIENT
↓
ENDPOINT
↓
AUTH
↓
VALIDATION
↓
CONTROLLER
↓
SERVICE
↓
DATABASE / AI

Each API specification should define:

- Purpose
 - Authentication
 - Authorization
 - Request
 - Response
 - Validation
 - Errors
 - Related entities
-

23. Security Documentation

Security documentation should cover:

AUTHENTICATION
↓
AUTHORIZATION
↓
TENANT ISOLATION
↓
DATA ACCESS
↓
AI ACCESS
↓
STORAGE
↓
AUDITABILITY

Security documentation should remain aligned with `rules.md` and `architecture.md`.

24. Development Documentation

Development documentation should help a developer understand:

- Where to implement a feature
- Which layer owns the logic
- Which APIs are involved
- Which database entities are involved
- Which AI integration is involved
- What documentation must be updated

The expected implementation path is:

FEATURE
↓
CLIENT
↓
API
↓
SERVER SERVICE
↓
DATABASE / AI

25. Documentation Update Rules

Whenever a significant architecture or product change is made, the appropriate documentation should be updated.

Examples:

New Product Feature

Update:

prd.md
architecture.md
phases.md

Database Change

Update:

architecture.md
docs/database
memory.md

UI Change

Update:

design.md
docs/frontend
memory.md

AI Pipeline Change

Update:

architecture.md
docs/ai
rules.md
memory.md

26. Documentation Consistency

The same concept should not have conflicting definitions across documents.

For example, if the product defines:

Candidate Score = 0–100

then:

- PRD
- Architecture
- Evaluation documentation
- Frontend reporting documentation

should all remain consistent.

27. Documentation Versioning Principle

Major architecture changes should be recorded in the project history.

A change should capture:

WHAT CHANGED



WHY IT CHANGED



WHAT SYSTEMS ARE AFFECTED



WHAT DOCUMENTS WERE UPDATED

This is especially important for:

- Database changes
 - AI provider changes
 - Authentication changes
 - Interview engine changes
 - Evaluation changes
 - Multi-tenant architecture changes
-

28. Documentation and Project Memory

memory.md should act as the bridge between permanent specifications and active development.

PERMANENT SPECIFICATION



IMPLEMENTATION



CURRENT STATE



memory.md

For example:

PRD says:

"Real-time AI interview required."

memory.md records:

"Real-time interview engine implemented through WebSocket session architecture; STT integration pending."

This keeps project status separate from product requirements.

29. Documentation and Development Phases

The roadmap should connect documentation to implementation.

PHASE



FEATURES



IMPLEMENTATION



TESTING



DOCUMENTATION UPDATE



PHASE COMPLETE

A phase should not be considered complete merely because code exists.

The related acceptance criteria must also be satisfied.

30. Documentation Review Process

Before marking a major feature complete:

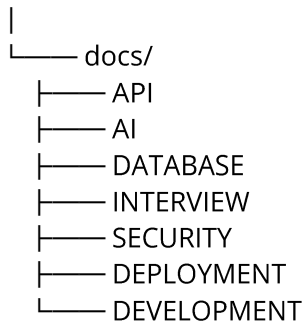
```
IMPLEMENTATION
↓
FUNCTIONAL CHECK
↓
SECURITY CHECK
↓
DATA CHECK
↓
DOCUMENTATION CHECK
↓
ACCEPTANCE CRITERIA
↓
COMPLETE
```

This prevents documentation from becoming outdated.

31. Repository Documentation Map

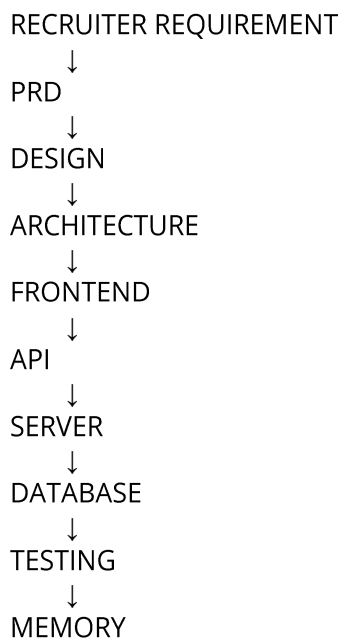
The complete documentation layer can be represented as:

```
QUALIFYAI/
|
|—— README.md
|
|—— prd.md
|
|—— architecture.md
|
|—— design.md
|
|—— memory.md
|
|—— phases.md
|
|—— rules.md
|
|—— document/
|   └── Formal Project Artifacts
```

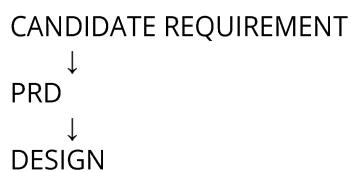


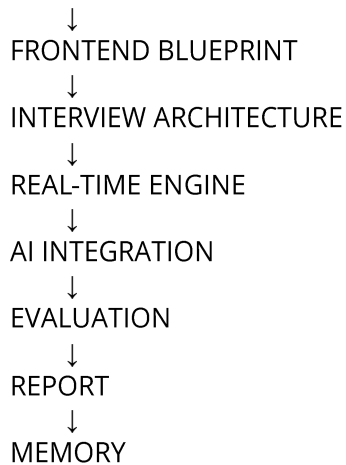
32. Documentation Flow for Recruiter Features

A recruiter feature should follow:

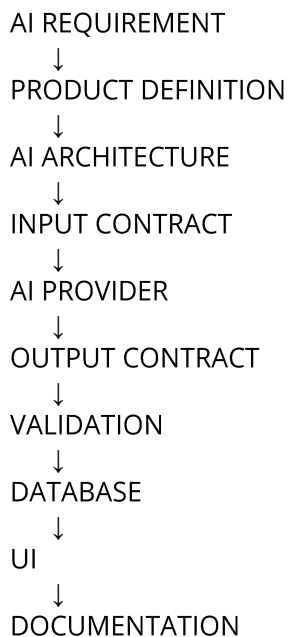


33. Documentation Flow for Candidate Features

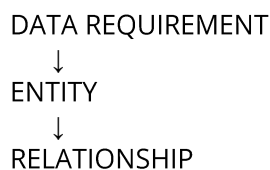


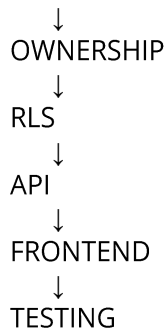


34. Documentation Flow for AI Features

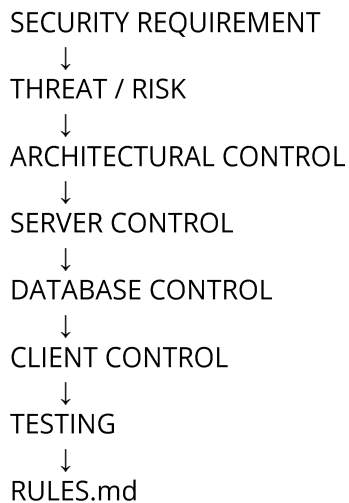


35. Documentation Flow for Database Features





36. Documentation Flow for Security Features



37. Technical Decision Documentation

Important technical decisions should be recorded when they materially affect the architecture.

Examples include:

- Why PostgreSQL is used
- Why Prisma is used
- Why WebSockets are required
- Why AI providers are abstracted
- Why evaluation is separated from conversation generation

- Why tenant isolation is enforced at multiple layers
- Why facial-expression scoring is excluded

A technical decision should contain:

DECISION
RATIONALE
IMPACT
ALTERNATIVES
STATUS

38. Documentation Boundaries

Documentation should not duplicate application code.

For example:

architecture.md should explain:

INTERVIEW SERVICE
↓
STT
↓
LLM
↓
TTS

It should not contain the complete implementation code.

Similarly, prd.md should explain what the feature must accomplish, not how every function is implemented.

39. Source-of-Truth Rules

When two documents appear to conflict, the following hierarchy should be used:

CURRENT APPROVED PRODUCT DECISION
↓
PRD / APPROVED SPECIFICATION
↓
ARCHITECTURE
↓

IMPLEMENTATION DOCUMENTATION



MEMORY / DEVELOPMENT NOTES

memory.md records current state but should not silently redefine the product requirements.

40. Future Documentation Expansion

As QualifyAI grows, additional technical documentation may be introduced under docs/.

Potential areas include:

```
docs/  
|  
├── api/  
├── ai/  
├── database/  
├── interview/  
├── security/  
├── deployment/  
├── integrations/  
└── development/
```

These are supporting documents and do not replace the seven root source-of-truth documents.

41. Documentation Acceptance Criteria

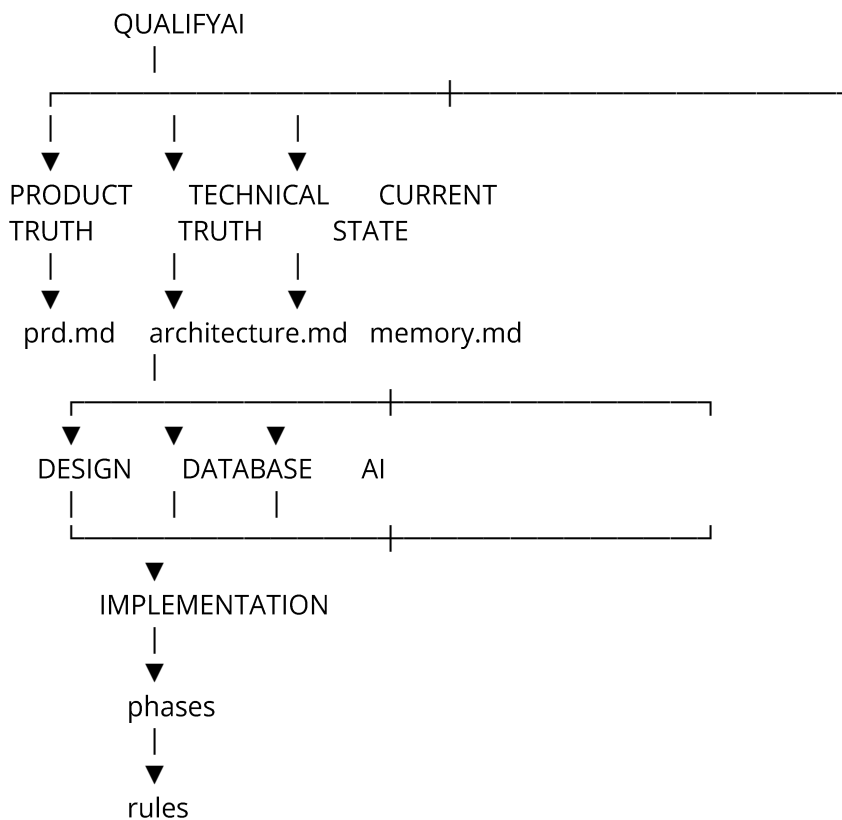
The documentation architecture is complete when:

- README explains the project.
- PRD defines product requirements.
- Architecture defines technical architecture.
- Design defines UX/design rules.
- Memory tracks current state.
- Phases define implementation roadmap.
- Rules define engineering invariants.
- Formal documents are separated under document/.
- Supporting technical documentation is separated under docs/.

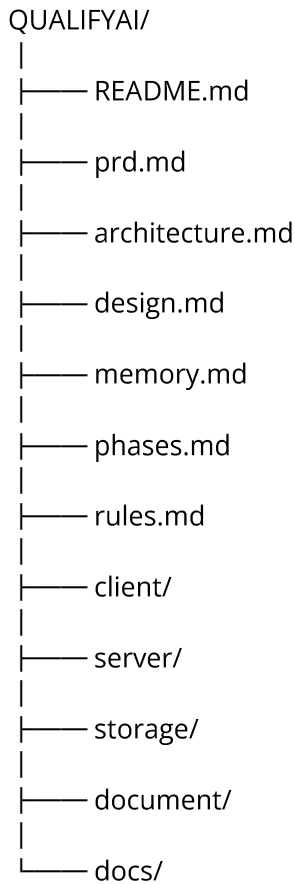
- AI architecture is documented.
 - Interview architecture is documented.
 - Database architecture is documented.
 - Security architecture is documented.
 - Major features remain traceable from requirement to implementation.
 - Documentation does not unnecessarily duplicate source code.
 - Significant architectural decisions are recorded.
-

42. Final QualifyAI Documentation Architecture

The complete documentation system is:



43. Final Repository Documentation Structure

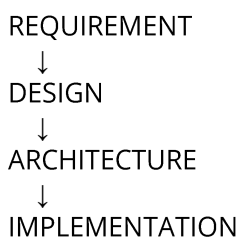


This preserves the required separation between:

- Application code
- Storage
- Formal project documents
- Living engineering documentation
- Permanent root specifications

44. Final Documentation Principle

QualifyAI documentation should always maintain the following chain:



↓
DATABASE / AI / SERVICES
↓
TESTING
↓
CURRENT STATE

No major feature should exist as an isolated implementation without a corresponding product and architectural explanation.

45. Final Definition

QualifyAI Documentation Architecture provides the organizational layer that connects the complete product lifecycle—from product requirements and UX design to system architecture, database design, AI interview processing, security, implementation phases, and current development state.

The documentation system is built around seven permanent root documents:

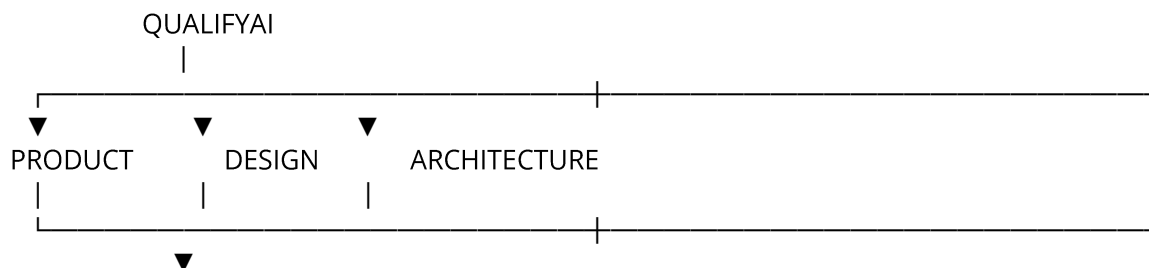
README.md
prd.md
architecture.md
design.md
memory.md
phases.md
rules.md

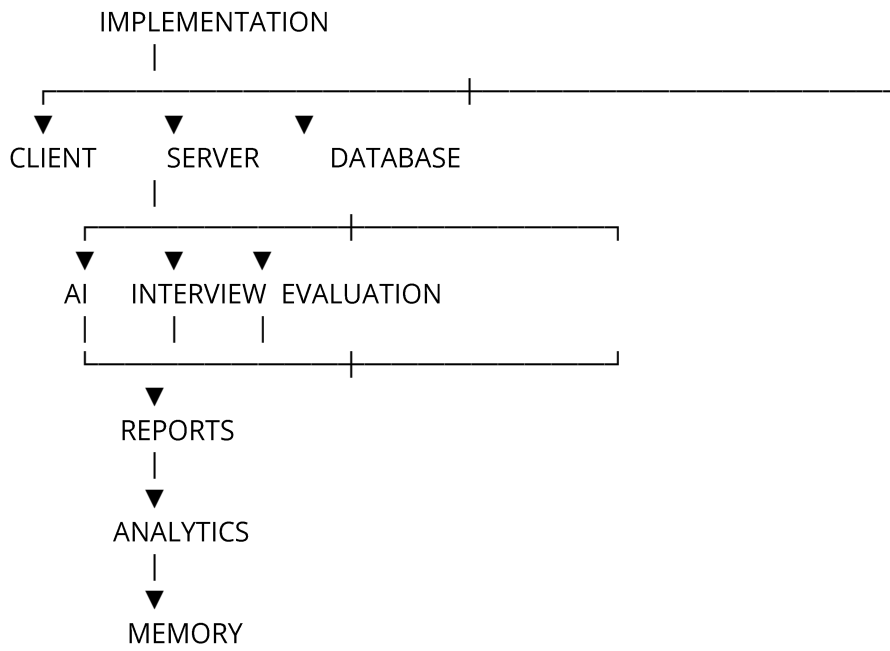
supported by:

document/
docs/

Together, these establish a structured source-of-truth system for the QualifyAI platform.

The final documentation relationship is:





Document 6 — QualifyAI Project Documentation & Reference Structure — Complete.